

Rapport — Projet Final — Data Engineering II (DE2)

Pipeline complet ETL • Streaming • Text • Graph • LLM Readiness

Présentation générale

Le projet final constitue l'intégration de tous les travaux réalisés durant le cours de Data Engineering II. Il s'agit de construire un pipeline de données complet de bout en bout, organisé en couches Bronze → Silver → Gold selon l'architecture Medallion, complété par un pipeline de streaming, un pipeline de traitement de texte avec index inversé, un workload itératif de PageRank sur graphe de co-picks de héros, et enfin une étape de préparation de données pour un LLM. Tout le projet tourne sous la session Spark nommée `de2-project`, alimentée par un fichier de configuration YAML `de2_project_config.yml` qui centralise tous les paramètres (chemins, seuils, paramètres de streaming, de texte, d'itération et de SLO).

Recours à l'IA

Conception de l'architecture Medallion : Au démarrage du projet, j'ai demandé à Claude d'expliquer concrètement la différence entre les couches Bronze, Silver et Gold dans une architecture Medallion. Il m'a expliqué que Bronze correspond aux données brutes déposées sans modification (immutables, potentiellement mal typées), Silver correspond aux données nettoyées, typées et déduplicées avec un contrat de schéma strict, et Gold correspond aux tables analytiques pré-agrégées et partitionnées, optimisées pour les requêtes en lecture. Il a insisté sur le fait que chaque couche doit être matérialisée physiquement (et non calculée à la volée) pour garantir reproductibilité et auditabilité.

Choix du format de stockage par couche : J'ai demandé à Claude pourquoi Bronze est écrit en CSV alors que Silver et Gold sont écrits en Parquet. Il m'a expliqué que Bronze conserve les données dans leur format d'origine pour garantir la fidélité à la source et permettre une relecture ultérieure si le schéma de nettoyage change. Silver et Gold bénéficient de Parquet pour ses avantages en lecture (encodage en colonnes, compression, pushdown de prédicats), ce qui est crucial pour les couches qui seront interrogées fréquemment par des jobs analytiques.

Utilisation du fichier de configuration YAML : Le fait de centraliser tous les paramètres dans `de2_project_config.yml` (watermark, trigger, stop-words, seuils SLO, etc.) était une décision de design que j'ai validée avec Claude. Il m'a confirmé que c'est une bonne pratique en data engineering : elle permet de modifier les paramètres sans toucher au code, de versionner la configuration séparément du notebook, et de réexécuter le pipeline avec des paramètres différents en changeant un seul fichier.

Conception de la fonction `log_metric` : J'ai demandé à Claude comment structurer un système de logging de métriques cohérent à travers toutes les étapes du pipeline. Il m'a suggéré d'utiliser une liste Python (`METRICS`) accumulée tout au long du notebook, avec une fonction `log_metric(stage, task, name, value, notes)` pour normaliser l'enregistrement, et d'écrire le CSV final seulement en fin de pipeline. Cette approche évite d'avoir à rouvrir et ré-écrire le fichier à chaque étape.

Compréhension du `xxhash64` pour la déduplication LLM : Dans la section LLM Data Readiness, la déduplication utilise `F.xxhash64("text")` plutôt qu'une comparaison directe sur le texte. J'ai demandé à Claude pourquoi. Il m'a expliqué que comparer directement des colonnes de texte long est très coûteux en mémoire et en CPU car Spark doit sérialiser et comparer des chaînes potentiellement très longues. Le hash `xxhash64` réduit chaque texte à un entier 64 bits, rendant la comparaison et le `dropDuplicates` beaucoup plus efficaces. Il a cependant précisé qu'un hash peut produire des collisions (deux textes différents produisant le même hash), bien que pour `xxhash64` la probabilité soit infime pour des volumes de données de cette taille.

Interprétation des vérifications SLO : La section 9 du notebook implémente un SLO check comparant les métriques mesurées à des seuils définis dans la configuration. J'ai demandé à Claude ce qu'est un SLO dans ce contexte et comment l'interpréter. Il m'a expliqué qu'un Service Level Objective est un seuil de qualité défini à l'avance : si le ratio Parquet/CSV dépasse un certain seuil, le pipeline est considéré comme inefficace en termes de compression ; si le taux de passages des filtres LLM est trop bas, la qualité des données est insuffisante. Cette approche permet d'automatiser la validation de la qualité sans inspection manuelle.

Aide pour construire les tables Gold partitionnées : La cellule Gold partitionne les tables par les colonnes définies dans `CFG["layout"]["partition_by"]`. J'ai demandé à Claude quel critère choisir pour la partition. Il m'a recommandé de partitionner par `region` car c'est une dimension de faible cardinalité (peu de valeurs distinctes) très fréquemment utilisée comme filtre dans les requêtes analytiques esports. Une faible cardinalité garantit des partitions de taille raisonnable et évite le problème de "small files" qui dégraderait les performances de Parquet.

Étape 1 — Bronze : ingestion des données brutes

La couche Bronze constitue la première étape du pipeline. Les fichiers CSV bruts sont lus depuis le répertoire de source défini dans la configuration via un glob pattern, sans modification du contenu, avec inférence du schéma uniquement pour inspection. Le nombre de lignes brutes est compté et le résultat est écrit tel quel dans le répertoire Bronze.

L'intérêt de cette approche est l'immuabilité : les données brutes sont conservées exactement telles qu'elles ont été reçues, permettant de rejouer les étapes de nettoyage ultérieures avec des règles différentes sans avoir à réingérer la source. Les métriques

enregistrées à cette étape sont le nombre de lignes, le poids sur disque en octets et le temps d'exécution.

Étape 2 — Silver : nettoyage, typage et déduplication

La couche Silver applique un contrat de schéma strict sur les données Bronze. Chaque colonne reçoit un type explicite : `match_id` en `IntegerType`, `match_end_time` converti en `TimestampType` via `F.to_timestamp`, les colonnes textuelles en `StringType`, et les métriques numériques (`kills`, `deaths`, `duration_sec`) en `IntegerType`. Les lignes avec des valeurs nulles dans les colonnes critiques (`match_id`, `match_end_time`, `team`, `hero`) sont supprimées via `dropna`. Les doublons sont supprimés sur la clé naturelle (`match_id`, `hero`) via `dropDuplicates`.

Le résultat est écrit en Parquet dans le répertoire Silver. Les métriques enregistrées incluent le nombre de lignes Silver, le footprint Parquet en octets, le temps d'exécution et le nombre de lignes supprimées (violation de contrat ou doublons). Le plan physique de la transformation est sauvegardé dans `proof/plan_etl.txt`.

Étape 3 — Gold : tables analytiques partitionnées

Deux tables Gold sont construites à partir de Silver. La première, `q1_team_perf`, agrège les performances par (`region`, `patch_version`, `team`) : nombre de picks, moyenne de kills, moyenne de deaths et durée moyenne de match. La seconde, `q2_hero_picks`, agrège par (`region`, `hero`) : nombre de picks et moyenne de kills.

Les deux tables sont écrites en Parquet partitionné par la configuration `partition_by` (typiquement `region`). Le partitionnement physique par région permet à Spark de ne lire que les partitions pertinentes lors d'une requête filtrée sur la région, ce qui évite un scan complet des données — mécanisme connu sous le nom de partition pruning. Les métriques enregistrées sont le nombre de lignes de chaque table, le footprint total de la couche Gold et le temps d'exécution.

Étape 4 — Pipeline de Streaming

Le pipeline de streaming reprend la structure développée en Lab 1 et Assignment 1, mais piloté par le fichier de configuration YAML qui définit le watermark, la durée de fenêtre, l'intervalle de trigger et le nombre de fichiers maximum par trigger. Les données sont lues depuis un répertoire `landing/` en JSON avec le même schéma que les labs.

L'agrégation applique un watermark sur `match_end_time` et regroupe par fenêtre temporelle et par équipe, calculant le nombre d'événements, la moyenne des kills et la durée

moyenne. Le résultat est écrit en Parquet en mode `append` vers `outputs/project/streaming`. Le plan de streaming est sauvegardé dans `proof/plan_streaming.txt` et le dernier `query.lastProgress` est sauvegardé dans `proof/query_progress.json`.

Les métriques enregistrées depuis `lastProgress` sont le nombre de lignes d'input du dernier batch, le `processedRowsPerSecond`, le `triggerExecution_ms`, et le nombre de lignes dans l'état interne. Le nombre de lignes dans le sink Parquet final est aussi enregistré.

Étape 5 — Pipeline de Traitement de Texte et Index Inversé

Le pipeline de texte est identique dans sa logique aux Lab 2 et Assignment 2, mais intégré dans le pipeline global avec la configuration YAML pour les stop-words et les termes de requête. Le corpus esports est normalisé (minuscules, suppression de la ponctuation), tokenisé, filtré des stop-words, et l'index inversé est construit par `groupBy("token")` avec `collect_list("doc_id")` et `count("*")`.

L'index est écrit en double format : Parquet dans `outputs/project/text/parquet` et CSV (avec `concat_ws` pour aplatir le tableau `doc_ids`) dans `outputs/project/text/csv`. Le plan d'exécution est sauvegardé dans `proof/plan_index_build.txt`.

La mesure de latence de requête est effectuée après mise en cache de l'index avec `idx.cache()` suivi de `idx.count()` pour forcer la matérialisation, garantissant des mesures de latence de lookup pur sans I/O disque. Le plan de la requête de lookup est sauvegardé dans `proof/plan_query.txt`.

Les métriques enregistrées couvrent le nombre de documents, les tokens avant et après filtrage des stop-words, le nombre de termes uniques, le temps de construction de l'index, les latences de requête pour chaque terme configuré, et le footprint Parquet vs CSV avec le ratio.

Étape 6 — Workload Itératif : PageRank sur graphe de co-picks

Cette étape est la plus technique du projet. Le graphe de co-picks est construit depuis la couche Silver (plutôt que depuis un fichier CSV dédié comme dans les labs), en lisant les colonnes (`match_id`, `hero`) et en effectuant une auto-jointure pour générer les paires de héros ayant joué le même match.

L'algorithme PageRank est encapsulé dans une fonction `run_pagerank(edges_in, ranks_in, out_deg, label)` réutilisée pour les deux runs, ce qui garantit que les deux expériences sont rigoureusement comparables en termes de code. Le checkpoint est utilisé

à chaque itération (`new_ranks = new_ranks.checkpoint()`) pour briser le lineage, approche apprise suite à l'erreur rencontrée en Lab 3.

Le Run 1 utilise le partitionnement par défaut de Spark. Le Run 2 repartitionne explicitement les arêtes par `src` et les rangs par `id` en `N_PART` partitions (défini dans la configuration). Les plans physiques avant et après repartitionnement sont sauvegardés dans `proof/plan_iterative_before.txt` et `proof/plan_iterative_after.txt`.

Les métriques enregistrées incluent le nombre de sommets et d'arêtes du graphe, le temps par itération pour les deux runs, le temps total et le speedup calculé comme `total_run1 / total_run2`. Les rangs finaux des deux runs sont écrits en Parquet dans `outputs/project/iterative/`.

Étape 7 — LLM Data Readiness

Cette étape constitue l'originalité du projet final par rapport aux labs. L'objectif est de préparer un corpus de haute qualité utilisable pour fine-tuner ou évaluer un LLM dans un domaine esports.

Deux sources de texte sont combinées par `unionByName` : le corpus de textes esports (lore de héros, patch notes, commentaires de matchs) et les textes de commentaire extraits de la couche Silver (`commentary_text`). Ces deux sources sont unifiées dans un DataFrame avec les colonnes `doc_id` et `text`.

Trois filtres de qualité sont appliqués successivement. Premièrement, suppression des lignes avec `text` nul. Deuxièmement, suppression des textes trop courts (sous le seuil `min_text_length` défini dans la configuration). Troisièmement, déduplication par hash du contenu avec `F.xxhash64("text")` pour éliminer les doublons sémantiques sans comparer les textes complets.

Le corpus curé est enrichi de métadonnées : `source` (la source du document), `version` (la version du projet), `track` (la track A — Esports), `curated_at` (timestamp de curation) et `text_len` (longueur en caractères). Il est ensuite écrit en Parquet dans `outputs/project/llm_ready`.

Les métriques enregistrées sont le nombre de records bruts, le nombre de records après filtrage, le taux de passage (pass rate), le seuil de longueur minimale appliqué et le footprint Parquet du corpus curé.

Étape 8 — Fichier de métriques global

Toutes les métriques accumulées tout au long du pipeline dans la liste **METRICS** sont écrites dans un fichier CSV unique **project_metrics_log.csv** avec les colonnes **run_id**, **stage**, **task**, **metric_name**, **metric_value**, **notes** et **timestamp**. Ce fichier constitue la preuve documentaire complète de l'exécution du pipeline, couvrant les étapes ETL, streaming, text, itératif, LLM et SLO.

Étape 9 — Vérification des SLOs

La dernière étape active du notebook vérifie que les métriques mesurées respectent les objectifs de service définis dans la configuration. Deux SLOs sont vérifiés. Le premier porte sur le ratio Parquet/CSV de l'index inversé : il doit être inférieur ou égal au seuil **parquet_csv_ratio_max**. Le résultat attendu est PASS avec un ratio mesuré de 13.92%, bien en dessous de tout seuil raisonnable. Le second porte sur le taux de passage des filtres LLM : il doit être supérieur ou égal au seuil **llm_quality_pass_rate_min**. Le résultat dépend du corpus et du seuil configuré, mais reflète la qualité intrinsèque des textes esports qui sont généralement bien formés.

Les résultats des SLO checks (PASS ou FAIL) sont eux-mêmes enregistrés dans le fichier de métriques lors de la réécriture finale du CSV, garantissant une traçabilité complète.

Synthèse et apprentissages

Ce projet final a permis d'intégrer les quatre grands thèmes du cours — ETL en couches, streaming, traitement de texte et workload itératif — dans un pipeline cohérent et automatisé. L'usage d'un fichier de configuration YAML a facilité la reproductibilité : en modifiant un seul fichier, on peut ajuster le watermark, le nombre d'itérations PageRank, les stop-words ou les seuils SLO sans toucher au code du notebook.

L'ajout de la couche LLM Data Readiness constitue une ouverture concrète vers les cas d'usage industriels actuels : les données traitées par un pipeline Spark peuvent alimenter directement un processus de fine-tuning ou d'évaluation d'un modèle de langage, à condition d'avoir appliqué des filtres de qualité rigoureux (longueur minimale, déduplication par hash) et d'avoir enrichi les documents de métadonnées de traçabilité (source, version, timestamp).